Vérifiez vos programmes Rust avec Creusot

Journée 2025 du GT LVP – Strasbourg

Jacques-Henri Jourdan

13 novembre 2025

Program verification is hard.

Program verification is hard.

But some tasks can be made easier by using the right tools.

The choice of a good programming language is important.

Verification of low-level software

Verification of low-level software is particularly difficult.

This is because they need control:

- over memory layout;
- over when the memory is allocated/deallocated;
- over what computation is done and when...

and many good verification languages/tools don't offer this control!

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

What spec?

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
// Ensures: dst[0..len] = old(src[0..len])
// Ensures: src[0..len] = old(src[0..len])
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

Wrong!

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
// Requires: distinct_memory(dst[0..len], src[0..len])
// Ensures: dst[0..len] = old(src[0..len])
// Ensures: src[0..len] = old(src[0..len])
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

We need to add non-aliasing hypotheses between all pairs of pointers.

- Trivial, but tedious to write and prove.
- Badly handled by automatic provers.

Rust



A language initially developed by Mozilla for rewriting parts of Firefox

High performance, high level of control with safe abstraction mechanisms

Our claim

Rust is well-suited for verification of low-level software.

Our claim

Rust is well-suited for verification of low-level software.

It has a rich type system.

- Safety guarantees.
  - No need to prove safety.
- Non-aliasing guarantees.
  - Helps to reason with pointers.
- Type traits (i.e., type classes): abstraction and genericity mechanism.
  - Easy to write reusable specifications.

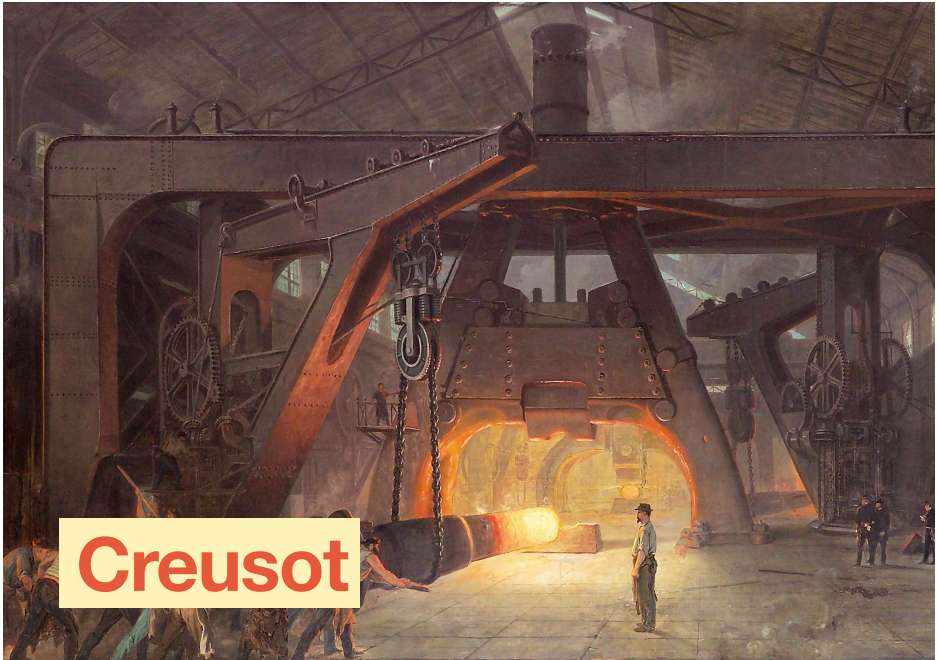
Our claim

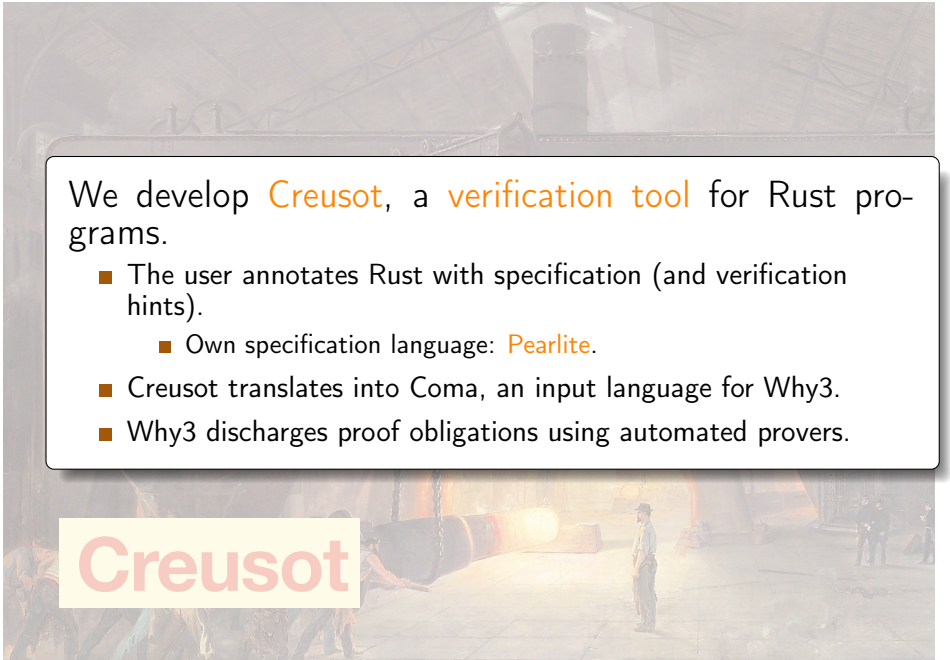
Rust is well-suited for verification of low-level software.

It has a rich type system.

- Safety guarantees.
  - No need to prove safety.
- Non-aliasing guarantees.
  - Helps to reason with pointers.
- Type traits (i.e., type classes): abstraction and genericity mechanism.
  - Easy to write reusable specifications.

It has tools with rich APIs on which we can build verification software.


Creusot



We develop Creusot, a verification tool for Rust programs.

- The user annotates Rust with specification (and verification hints).
  - Own specification language: Pearlite.
- Creusot translates into Coma, an input language for Why3.
- Why3 discharges proof obligations using automated provers.

Creusot

Outline

1. Our big secret for handling pointers easily.
2. Demo: proving gnome sort with Creusot.
3. Ghost code to prove programs with non-trivial aliasing discipline.

Rust is a functional* programming language

... and this greatly helps verification.

*some squinting required.

`Box<T>` is Rust's type for owned pointers.

- I.e., pointers that are allocated on the heap ($\simeq$ `malloc`).
- Type system guarantee: no alias.

`Box<T>` is Rust's type for owned pointers.

- I.e., pointers that are allocated on the heap ($\simeq$ `malloc`).
- Type system guarantee: no alias.

```
fn incr(x: Box<u64>, y: Box<u64>)
      -> (Box<u64>, Box<u64>) {
  *x += *y;
  do_stuff();
  (x, y)
}
```

```
let incr x y =
  let x = x + y in
  do_stuff();
  (x, y)
```

Because there is no alias, we know nobody else can mutate `x` and `y`.

Owned pointers functionally behave like their content.

`&T` is Rust's type for temporarily immutable pointers.

`&T` is Rust's type for temporarily immutable pointers.

```
fn f() -> i32 {
  let x = 12;
  let a : &i32 = &x;
  let b : &i32 = a;  // allowed alias
  do_stuff(a, b);
  let y = *a + *b;
  y
}
```

```
let f () =
  let x = 12 in
  let a = x in
  let b = a in
  do_stuff(a, b);
  let y = a + b in
  y
```

Because of immutability, shared borrows `&T` functionally behave like their contents.

The reason all this works is that Rust enforces an invariant:

Mutation XOR aliasing

i.e., a memory location that is pointed to by several pointers cannot be mutated.

Consequence: there is no "action at a distance", and it is easy to track changes of variables.

The reason all this works is that Rust enforces an invariant:

Mutation XOR aliasing

i.e., a memory location that is pointed to by several pointers cannot be mutated.

Consequence: there is no "action at a distance", and it is easy to track changes of variables.

We can see such a program as a functional program.

- Creusot: translates Rust programs into a functional program.

Mutable borrows

Rust also has a type `&mut T` of mutable borrows.
Gives temporary mutable access to a variable, without aliasing.

- (Disables direct access to the variable, so the aliasing rule is still followed.)

Mutable borrows

Rust also has a type `&mut T` of mutable borrows.
Gives temporary mutable access to a variable, without aliasing.

- (Disables direct access to the variable, so the aliasing rule is still followed.)

Examples:

```
// Vec<T>: extensible array.

// Add a new element
fn push<T>(v: &mut Vec<T>, value: T) { ... }

// Get a mutable reference to the element at index idx
// e.g., to mutate it.
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

```
fn push<T>(v: &mut Vec<T>, value: T) { ... }
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

How do we translate thes uses of these functions?

```
fn f() {
   let v = vec![1, 2];

   push(&mut v, 2);

   *index_mut(&mut v, 2) += 1;
   // I.e., v[2] += 1;
}
```

```
let f =
  let v = [1, 2];

  ??? push(v, 2); ???

  ??? index_mut(v, 2) + 1 ???
```

Prophecies

Our trick to solve this problem is to use a prophecy.

- a "variable" that we can read *before* assigning.
  i.e., its value depends on the future of the execution.

A typical proof that uses a prophecy variable:

1. Creation of the prophecy early in the program's execution
   - A value is fixed. We don't have any information on it.
2. Some proof steps which depend on the value of the prophecy.
3. Resolution
   - We *assign* a value to the prophecy.
   - In the proof, we learn that its (past) value is equal to that value.

Prophecies and mutable borrows

`x: &mut T` is translated into a pair:

- of the current value noted `*x`,
- and the prophetized final value of the borrow, noted `^x`.

The prophecy is used to assign its new value to the borrowed variable.

Prophecies and mutable borrows

Example: `push`

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);




   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in


  push(v_bor, 2);


  let v = v_proph in


  v
```

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);




   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in


  push(v_bor, 2);
  (* We now know that "fst v_bor == v ++ [2]" *)

  let v = v_proph in

  (* So we can deduce that "v == [1, 2, 2]", as expected! *)
  v
```

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);




   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in
  let v = v_proph in

  push(v_bor, 2);




  v
```

Prophecies and mutable borrows

Example: `index_mut`

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

Prophecies and mutable borrows

Example: index_mut

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2, 2];

   let v_bor = &mut v;

   let x = index_mut(v_bor, 2);

   *x += 1;

   v
}
```

```
let f =
  let v = [1, 2, 2];

  let v_bor = (v, any) in
  let v = snd v_bor in

  let x = index_mut(v_bor, 2);

  let x = (fst x + 1, snd x) in
  (* x will no longer be used, we resolve its prophecy. *)
  assume (fst x = snd x);

  v
```

Prophecies and mutable borrows

Example: index_mut

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2, 2];

   let v_bor = &mut v;

   let x = index_mut(v_bor, 2);

   *x += 1;

   v
}
```

```
let f =
  let v = [1, 2, 2];

  let v_bor = (v, any) in
  let v = snd v_bor in

  let x = index_mut(v_bor, 2);
  (* We learn "fst x = 2" and "v = [1, 2, snd x]". *)

  let x = (fst x + 1, snd x) in

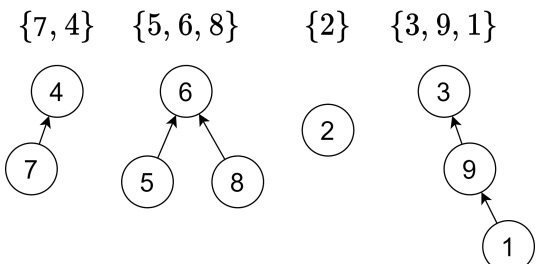
  assume (fst x = snd x);
  (* We learn "3 = fst x = snd x = v[2]" *)
  v (* == [1, 2, 3] *)
```

Demo time!

Next example: union-find

Maintains a partition of a set of elements. Two operations: `union` and `find`.

Represented with trees, with pointers to the root:



Problem: “action at a distance”

Consider the piece of code:

```
let (a, b, c, d) = (UF::elem(), UF::elem(),  UF::elem(), UF::elem());
UF::union(a, b);
UF::union(c, d);
assert!(UF::find(a) != UF::find(c));
UF::union(b, d);
assert!(UF::find(a) == UF::find(c));
```

Problem: “action at a distance”

Consider the piece of code:

```
let (a, b, c, d) = (UF::elem(), UF::elem(),  UF::elem(), UF::elem());
UF::union(a, b);
UF::union(c, d);
assert!(UF::find(a) != UF::find(c));
UF::union(b, d);
assert!(UF::find(a) == UF::find(c));
```

The value of `UF::find(a) == UF::find(c)` changed “at a distance”!

Problem: “action at a distance”

Consider the piece of code:

```
let (a, b, c, d) = (UF::elem(), UF::elem(),  UF::elem(), UF::elem());
UF::union(a, b);
UF::union(c, d);
assert!(UF::find(a) != UF::find(c));
UF::union(b, d);
assert!(UF::find(a) == UF::find(c));
```

The value of `UF::find(a) == UF::find(c)` changed “at a distance”!

Hidden global state.

Specification using ghost state

```
#[ensures(result.domain().is_empty())]
pub fn new() -> Ghost<UF> { ... }


#[ensures(!uf.in_domain(result))]
#[ensures((^uf).domain() == uf.domain().insert(result))]
#[ensures((^uf).roots_map() == uf.roots_map().set(result, result))]
pub fn elem(mut uf: Ghost<&mut UF>) -> Element { ... }


#[requires(uf.in_domain(elem))]
#[ensures(result == uf.root(elem))]
#[ensures(uf.unchanged())]
pub fn find(mut uf: Ghost<&mut UF>, elem: Elem) -> Elem { ... }


#[requires(uf.in_domain(x) @@ uf.in_domain(y))]
#[ensures(uf.domain_unchanged())]
#[ensures(result == uf.root(x) || result == uf.root(y))]
#[ensures(forall<z> uf.in_domain(z) ==>
    (^uf).root(z) ==
        if uf.root(z) == uf.root(x) || uf.root(z) == uf.root(y) { result }
        else { uf.root(z) }
)]
pub fn union(mut uf: Ghost<&mut UF>, x: Elem, y: Elem) { ... }
```

Specification using ghost state

```
#[ensures(result.domain().is_empty())]
pub fn new() -> Ghost<UF> { ... }

#[ensures(
#[ensures(
#[ensures(
pub fn elem

#[requires(
#[ensures(r
#[ensures(u
pub fn find

#[requires(
#[ensures(uf.domain_unchanged())]
#[ensures(result == uf.root(x) || result == uf.root(y))]
#[ensures(forall<z> uf.in_domain(z) ==>
    (^uf).root(z) ==
        if uf.root(z) == uf.root(x) || uf.root(z) == uf.root(y) { result }
        else { uf.root(z) }
)]
pub fn union(mut uf: Ghost<&mut UF>, x: Elem, y: Elem) { ... }
```

The type `Ghost<T>` is compiled to the unit type: no runtime cost.

But it is transparent during verification:

- It has a value of type `T`.
- It owns the same resources as `T`.

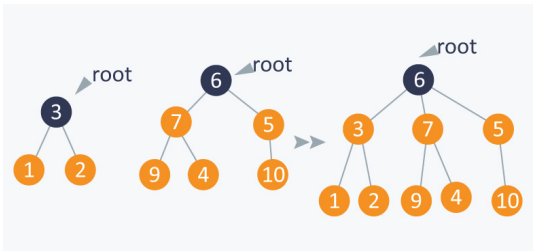
Aliasing in union-find

Good, we know how to specify union-find.
How should we program and verify it?

Aliasing in union-find

Good, we know how to specify union-find.
How should we program and verify it?

Problem: union-find is inherently an aliased data structure, with mutation:



Even more so with path compression!

Avoiding the aliasing discipline in Rust

It is not always possible to follow the aliasing discipline:

- We sometimes need unrestricted aliasing (not guarded by a syntactic lifetime).
- We sometimes need to mutate through an aliased pointer.

In these cases, Rust provides tools:

- Interior mutability: mutation under shared borrows allowed by some libraries.
  - Most often: safe.
- Raw pointers: unrestricted pointers (just like in C/C++).
  - Unsafe, may break all the type system guarantees.

Avoiding the aliasing discipline in Rust

It is not always possible to follow the aliasing discipline:

- We
- We

In these

- Inte
  -
- Raw pointers: unrestricted pointers (just like in C/C++).
  - Unsafe, may break all the type system guarantees.

But then our functional translation is broken (there are actions at a distance)!

How should we prove such programs?

Avoiding the aliasing discipline in Rust

It is not always possible to follow the aliasing discipline:

- We
- We

In these

- Inte
  - Raw pointers: unrestricted pointers (just like in C/C++).
    - Unsafe, may break all the type system guarantees.

But then our functional translation is broken (there are actions at a distance)!

How should we prove such programs?

Main idea: Rust type system tracks ownership.
Let's encode separation logic in Rust!

The `PtrOwn` type

`PtrOwn<T>`: ghost type representing ownership of a memory location.

In separation logic: equivalent of $p \mapsto v$.

```
impl<T> PtrOwn<T> {
    #[ensures(result.1.ptr() == result.0 && *result.1.val() == v)]
    pub fn new(v: T) -> (*mut T, Ghost<PtrOwn<T>>) { ... }
```

$\{\text{True}\}\ \texttt{ref}(v)\ \{\ell.\ \ell \mapsto v\}$

```
    #[requires(ptr == own.ptr())]
    #[ensures(*result == *own.val())]
    pub unsafe fn as_ref(ptr: *const T, own: Ghost<&PtrOwn<T>>) -> &T { ... }
```

$\{\ell \mapsto v\}\ !\ell\ \{w.\ w = v * \ell \mapsto v\}$

```
    #[requires(ptr as *const T == own.ptr())]
    #[ensures(*result == *own.val())]
    #[ensures((^own).ptr() == own.ptr())]
    #[ensures(*(^own).val() == ^result)]
    pub unsafe fn as_mut(ptr: *mut T, own: Ghost<&mut PtrOwn<T>>) -> &mut T { ... }
}
```

$\{\ell \mapsto v\}\ \ell \leftarrow w\ \{\ell \mapsto w\}$

`PtrOwn` in union-find proof

The definition of `UF` contains `PtrOwn` values for the union-find blocks:

```
pub struct UF {
    // Which "pointers" are involved
    domain: Snapshot<FSet<Elem>>,

    // Maps an element to its logical content (represented by the permission to access it).
    perms: FMap<Elem, PtrOwn<Node>>,

    // Map each element in [‘Self::domain‘] to its canonical representative.
    roots: Snapshot<Mapping<Elem, Elem>>,
}
```

Type invariant for `UF`

A type invariant is used to maintain well-formedness properties of `UF`:

```
pub struct UF {
    domain: Snapshot<FSet<Elem>>,
    perms: FMap<Elem, PtrOwn<Node>>,
    roots: Snapshot<Mapping<Elem, Elem>>,
}

impl Invariant for UF {
    #[logic(inline)]
    fn invariant(self) -> bool {
        pearlite! {
        forall<e> self.domain.contains(e) ==>
            self.perms.contains(e) &&
            self.perms[e].ptr() == e.0 as *const Node &&
            ...
    }
}
```

Example proof: `find`

```
#[requires(uf.in_domain(elem))]
#[ensures(result == uf.root(elem))]
#[ensures(uf.unchanged())]
pub fn find(mut uf: Ghost<&mut UF>, elem: Elem) -> Elem {
    ghost_let!(perm = uf.perms.get_ghost(&elem).unwrap());
    match unsafe { PtrOwn::as_ref(elem.0 as *const Node, perm) } {
        Node::Root { .. } => elem,
        &Node::Link(e) => {
            let root = find(ghost! {&mut **uf}, e);
            // path compression
            ghost_let!(mut perms = &mut uf.perms);
            unsafe {
                *PtrOwn::as_mut(
                    elem.0 as *mut Node,
                    ghost!(perms.get_mut_ghost(&elem).unwrap()),
                ) = Node::Link(root)
            };
            root
        }
    }
}
```

Not in this talk: Iris-style resources and invariants.

$$\text{GHOST-OP}\quad \boxed{a \cdot b}^{\gamma} \Leftrightarrow \boxed{a}^{\gamma} * \boxed{b}^{\gamma} \qquad \text{GHOST-VALID}\quad \boxed{a}^{\gamma} \Rightarrow \overline{\mathcal{V}}(a) \qquad \text{GHOST-UPDATE}\quad \frac{a \rightsquigarrow B}{\boxed{a}^{\gamma} \Rrightarrow_{\mathcal{E}} \exists b \in B.\, \boxed{b}^{\gamma}}$$

$$\text{HOARE-INV}\quad \frac{\{\triangleright P * Q_1\}\, e\, \{v. \triangleright P * Q_2\}_{\mathcal{E}\setminus\mathcal{N}} \qquad \text{atomic(e)} \qquad \mathcal{N} \subseteq \mathcal{E}}{\left\{\boxed{P}^{\mathcal{N}} * Q_1\right\} e \left\{v. \boxed{P}^{\mathcal{N}} * Q_2\right\}_{\mathcal{E}}}$$

Useful to define custom resources and protocols.

Iterators and closures

Not in this talk:

- Modular specification for iterators, including mutating iterators.
- Support for (effectful) closures, with specification inference.

Example (automated proof):

```
pub fn test_iter() {
    let mut v = vec![0, 1, 2, 3];
    let mut n = 0;
    let v2: Vec<i32> = v
        .iter_mut()
        .map_inv(|r: &mut i32, _prod: Snapshot<Seq<&mut i32>>| {
            proof_assert!(n@ == _prod.len());
            *r += 1;
            n += 1;
            *r + 1
        })
        .collect();
    proof_assert!(n@ == v@.len());
    proof_assert!(v@[1] == 2i32);
    proof_assert!(v2@[1] == 3i32)
}
```

Future work:

- Concurrency under the weak memory model of C++ 20
- Verification of interaction with high-level languages (e.g., OCaml GC)

Questions?

https://github.com/creusot-rs/creusot